

ВОПРОСЫ
для подготовки к экзамену по дисциплине
«ОБЪЕКТНО-ОРИЕНТИРОВАННЫЕ ТЕХНОЛОГИИ
ПРОГРАММИРОВАНИЯ И СТАНДАРТЫ ПРОЕКТИРОВАНИЯ»
(Часть 2)

для студентов специальности
«Программная инженерия»

1. Парадигмы объектно-ориентированного программирования: инкапсуляция, наследование и полиморфизм. Их сущность, практическое значение и примеры реализации.
2. Основные понятия ООП: класс и объект. Их взаимосвязь и различия.
3. Принципы модульного программирования и их сравнение с объектно-ориентированным подходом.
4. Инкапсуляция как фундаментальный принцип ООП и ее реализация с помощью модификаторов доступа.
5. Объект как динамический модуль и преимущества такого подхода.
6. Методы класса и их отличие от обычных процедур.
7. Назначение, виды конструкторов и деструкторов, порядок их вызова.
8. Свойства объектов: механизм getter/setter и преимущества перед открытыми полями.
9. Свойство-индексатор, его назначение и сценарии применения.
10. Принцип наследования в ООП: цели, синтаксис и создание производных классов.
11. Различия между переопределением (override) и сокрытием (new) методов.
12. Базовый класс ("прародитель") всех классов в языках Java и C#, его ключевые методы.

13. Преобразование типов: восходящее и нисходящее преобразование, операторы для их выполнения.
14. Полиморфизм и его связь с наследованием и виртуальными методами.
15. Виртуальные методы: принцип работы, использование таблицы виртуальных методов (vtable).
16. Абстрактные классы и методы, их назначение и ограничения.
17. Механизмы запрета наследования и переопределения методов в современных языках программирования.
18. Обработка исключительных ситуаций: механизм try-catch-finally и его преимущества.
19. Создание собственных классов исключений, построение иерархии исключений.
20. Защита ресурсов от утечки при возникновении исключений, роль деструкторов и блоков finally.
21. Делегаты и события: их сущность и роль в реализации паттерна Наблюдатель.
22. Практические примеры использования делегатов и событий для упрощения архитектуры приложений.
23. Интерфейсы, их отличие от абстрактных классов и механизм реализации.
24. Решение проблемы множественного наследования через множественную реализацию интерфейсов.
25. Совместимость интерфейсов и классов, проверка реализации интерфейса.
26. Шаблоны (обобщенное программирование): назначение и решаемые проблемы.
27. Объявление и использование шаблонных функций и классов.
28. Наложение ограничений на параметры шаблонов.
29. Атрибуты (аннотации): назначение и примеры использования.
30. Понятие и назначение паттернов проектирования, известные каталоги паттернов.

31. Порождающие паттерны: Одиночка, Фабричный метод, Абстрактная фабрика.
32. Структурные паттерны: Адаптер, Декоратор, Фасад.
33. Поведенческие паттерны: Наблюдатель, Стратегия, Состояние.
34. Особенности работы с объектами в стеке и куче в C++, операторы new и delete.
35. Конструкторы и деструкторы в C++: копирование, присваивание, правило трех/пяти.
36. Множественное наследование в C++, проблема ромбовидного наследования и виртуальное наследование.
37. Виртуальные методы в C++: синтаксис, механизм vptr и vtable.
38. Перегрузка операторов в C++, ее цели и ограничения.
39. Ссылки в C++ и их отличие от указателей.
40. Обработка исключений в C++, механизм try-catch.
41. Шаблоны в C++, их отличия от дженериков Java/C#, специализация шаблонов.
42. Особенности реализации ООП в Java и C#: общие черты и ключевые различия.
43. Управление памятью в Java/C#: сборщик мусора и его влияние на разработку.
44. Назначение и основные виды диаграмм UML: классов, вариантов использования, последовательности.
45. Основные элементы диаграммы классов UML: классы, атрибуты, методы, связи.
46. CASE-технологии: назначение, примеры средств и их роль в разработке ПО.
47. Rational Unified Process (RUP): основные принципы и связь с UML.
48. Принципы SOLID и их важность для создания качественного программного обеспечения.
49. Рефакторинг кода: сущность и примеры рефакторингов, улучшающих ООП-дизайн.
50. Возможности современных IDE для поддержки ООП: рефакторинг, навигация, генерация кода.

51. Модульное тестирование и его связь с принципами ООП, паттерн Mock Object.

52. Основные этапы жизненного цикла объектно-ориентированного проекта: анализ, проектирование, программирование.